

Enterprise .NET API Lesson Plan - Part 1

Question Canvas: ASP.NET Core + EF Core + SQLite + Identity

Baseline: .NET 10 LTS / Visual Studio Code / August 2026

Student outcome: follow every step in Visual Studio Code and finish with a working, testable project.

Lesson map

1. What you are building and why
 2. Install and verify prerequisites
 3. Create the repository and layered solution
 4. Add project references and NuGet packages
 5. Build the Domain layer
 6. Build the Application layer
 7. Build the Infrastructure layer with EF Core and Identity
 8. Build the API layer and enterprise middleware
 9. Create the SQLite database with migrations
 10. Run and test authentication and ownership
 11. Add integration-test scaffolding
 12. Architecture review and Part 2 handoff
- A. Endpoint reference
 - B. Troubleshooting

1. What you are building and why

You are building the backend for Question Canvas, a multi-user application in which each authenticated user owns projects, and each project contains one or more saved questions. Part 1 persists and retrieves the projects and questions. Part 2 will generate an image answer for a saved question.

1.1 Final Part 1 behavior

- A user can register, sign in, and sign out through ASP.NET Core Identity.
- An authenticated user can create a project with a Project Name.
- An authenticated user can add multiple Question values to a project.
- A user can list and open only projects that they own.
- An attempt to open another user's project returns 404, not data and not an ownership hint.
- Data persists to a real SQLite database through Entity Framework Core migrations.
- Queries use async LINQ operators such as Where, Include, OrderByDescending, AnyAsync, and SingleOrDefaultAsync.
- Unhandled exceptions are converted to consistent ProblemDetails responses and include a trace identifier.
- The API exposes liveness/readiness health checks and basic rate limiting.
- The solution contains a separate integration-test project.

1.2 Why the backend is multiple projects

Enterprise standards do not require a separate Git repository for every layer, but they do favor clear dependency boundaries. This lesson uses one backend solution with separate projects so HTTP concerns, business use cases, domain objects, and database/Identity concerns do not collapse into one project.

Project	Responsibility	May depend on
QuestionCanvas.Domain	Core entities and domain data	Nothing in this solution
QuestionCanvas.Application	Use cases, DTOs, interfaces, application exceptions	Domain
QuestionCanvas.Infrastructure	EF Core DbContext, Identity user, SQLite, LINQ repository	Application + Domain
QuestionCanvas.Api	Controllers, auth endpoints, middleware, DI composition root	Application + Infrastructure
QuestionCanvas.Api.IntegrationTests	HTTP-level verification	API

Architecture choice: We intentionally do not create a `GenericRepository<T>`. EF Core already behaves like a unit of work with repository-like `DbSet` access. The repository in this lesson is specific to the Question Canvas aggregate and exists to isolate persistence queries, not to wrap every EF Core method.

1.3 Authentication choice

For this browser-based React SPA, the lesson uses ASP.NET Core Identity cookie authentication. The browser owns the authentication cookie, JavaScript never stores a bearer token, and every protected project endpoint uses ASP.NET Core authorization. Part 2 can keep this exact security model.

1.4 Planned Part 2 image contract

Part 2 should add a protected generation endpoint beneath a specific saved question. The recommended response is binary image content (for example `image/png`), not a base64 string embedded in JSON. The React app can turn the response into a `Blob` or `ImageBitmap` and draw it into its canvas for pixel manipulation.

Future endpoint - do not implement in Part 1

```
POST /api/projects/{projectId}/questions/{questionId}/generate
Authorization: authenticated cookie
```

```
200 OK
Content-Type: image/png
<body is image bytes>
```

2. Install and verify prerequisites

Install these tools before creating files. Keep Visual Studio Code as the IDE; all .NET commands are run from its integrated terminal.

- .NET 10 SDK. .NET 10 is the current LTS line for this lesson.
- Visual Studio Code.
- C# Dev Kit extension for Visual Studio Code.
- SQLite viewer extension is optional but helpful. You can also inspect data using sqlite3 if installed.
- Git.
- A terminal with curl. On Windows, use curl.exe to avoid the PowerShell alias behavior.

Run in the VS Code terminal

```
dotnet --version
git --version
code --version
curl --version
```

Your dotnet version must begin with 10. A later servicing patch is fine. This lesson pins the Microsoft EF/ASP.NET packages to the .NET 10 servicing line. If a later 10.0.x patch is installed, keep all Microsoft.EntityFrameworkCore and ASP.NET Identity package versions on the same 10.0.x line.

Trust the local ASP.NET Core HTTPS development certificate

```
dotnet dev-certs https --trust
```

Linux note: The trust step varies by distribution. If your browser or curl does not trust the development certificate, follow the .NET development certificate guidance for your OS rather than disabling TLS in production settings.

Checkpoint: Do not continue until every item below is true.

- [] dotnet --version reports 10.x.
- [] VS Code can open a folder and the C# Dev Kit extension is enabled.
- [] The HTTPS development certificate is trusted or you know how you will handle it locally.

3. Create the repository and layered solution

Choose a parent folder where you keep source code, open it in VS Code, then run the following commands. The frontend folder is created now but remains empty until the React PDF.

Create solution and projects

```
mkdir EnterpriseQuestionCanvas
cd EnterpriseQuestionCanvas
mkdir backend
mkdir frontend
cd backend

dotnet new sln -n QuestionCanvas

dotnet new classlib -n QuestionCanvas.Domain -o src/QuestionCanvas.Domain -f net10.0
dotnet new classlib -n QuestionCanvas.Application -o src/QuestionCanvas.Application -f net10.0
dotnet new classlib -n QuestionCanvas.Infrastructure -o src/QuestionCanvas.Infrastructure -f net10.0
dotnet new webapi -n QuestionCanvas.Api -o src/QuestionCanvas.Api --use-controllers -f net10.0
dotnet new xunit -n QuestionCanvas.Api.IntegrationTests -o tests/QuestionCanvas.Api.IntegrationTests -f net10.0

dotnet sln QuestionCanvas.slnx add src/QuestionCanvas.Domain/QuestionCanvas.Domain.csproj
dotnet sln QuestionCanvas.slnx add src/QuestionCanvas.Application/QuestionCanvas.Application.csproj
dotnet sln QuestionCanvas.slnx add src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj
dotnet sln QuestionCanvas.slnx add src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
dotnet sln QuestionCanvas.slnx add tests/QuestionCanvas.Api.IntegrationTests/QuestionCanvas.Api.IntegrationTests.csproj
```

.NET 10 solution format: Starting with .NET 10, dotnet new sln creates QuestionCanvas.slnx by default. That is expected and supported by current .NET tooling.

From the EnterpriseQuestionCanvas root, open the whole repository rather than only the API project:

Open the repository root in the current VS Code window

```
cd ..  
code -r .
```

Delete the template files that are not part of this application:

- src/QuestionCanvas.Domain/Class1.cs
- src/QuestionCanvas.Application/Class1.cs
- src/QuestionCanvas.Infrastructure/Class1.cs
- src/QuestionCanvas.Api/Controllers/WeatherForecastController.cs if the template created it
- src/QuestionCanvas.Api/WeatherForecast.cs if the template created it
- tests/QuestionCanvas.Api.IntegrationTests/UnitTest1.cs

Checkpoint: Do not continue until every item below is true.

[] Explorer shows backend/src with four projects and backend/tests with one test project.

[] QuestionCanvas.slnx exists in backend.

[] The default Class1/WeatherForecast files are removed.

4. Add project references and NuGet packages

4.1 Project-to-project references

Run from EnterpriseQuestionCanvas/backend

```
cd backend  
  
dotnet add src/QuestionCanvas.Application/QuestionCanvas.Application.csproj reference  
src/QuestionCanvas.Domain/QuestionCanvas.Domain.csproj  
  
dotnet add src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj reference  
src/QuestionCanvas.Domain/QuestionCanvas.Domain.csproj  
dotnet add src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj reference  
src/QuestionCanvas.Application/QuestionCanvas.Application.csproj  
  
dotnet add src/QuestionCanvas.Api/QuestionCanvas.Api.csproj reference  
src/QuestionCanvas.Application/QuestionCanvas.Application.csproj  
dotnet add src/QuestionCanvas.Api/QuestionCanvas.Api.csproj reference  
src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj  
  
dotnet add tests/QuestionCanvas.Api.IntegrationTests/QuestionCanvas.Api.IntegrationTests.csproj reference  
src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
```

4.2 NuGet packages

The examples below use 10.0.11, the current .NET 10 servicing patch at the time this lesson was written. If you deliberately update later, update the Microsoft 10.0.x packages together and rerun the full test checklist.

Install packages

```
dotnet add src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj package  
Microsoft.EntityFrameworkCore.Sqlite --version 10.0.11  
dotnet add src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj package  
Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 10.0.11  
  
dotnet add src/QuestionCanvas.Api/QuestionCanvas.Api.csproj package Microsoft.EntityFrameworkCore.Design --  
version 10.0.11  
dotnet add src/QuestionCanvas.Api/QuestionCanvas.Api.csproj package  
Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore --version 10.0.11
```

```
dotnet add tests/QuestionCanvas.Api.IntegrationTests/QuestionCanvas.Api.IntegrationTests.csproj package
Microsoft.AspNetCore.Mvc.Testing --version 10.0.11
```

4.3 Pin dotnet-ef as a repository-local tool

A local tool manifest makes migrations reproducible for the team and avoids depending on whichever global dotnet-ef version happens to be installed.

Run in backend

```
dotnet new tool-manifest
dotnet tool install dotnet-ef --version 10.0.*
dotnet tool restore
dotnet ef --version
```

First clean build

```
dotnet restore
dotnet build QuestionCanvas.slnx
```

Checkpoint: Do not continue until every item below is true.

- [] Restore succeeds.
- [] Build succeeds before application code is added.
- [] dotnet ef --version reports a 10.0.x version.

5. Build the Domain layer

The Domain project contains the data your business cares about. It does not reference ASP.NET Core, EF Core, controllers, cookies, or SQLite.

5.1 Create the project entity

src/QuestionCanvas.Domain/Entities/QuestionProject.cs

```
namespace QuestionCanvas.Domain.Entities;

public sealed class QuestionProject
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Name { get; set; } = string.Empty;
    public string NormalizedName { get; set; } = string.Empty;
    public string OwnerUserId { get; set; } = string.Empty;
    public DateTimeOffset CreatedUtc { get; set; } = DateTimeOffset.UtcNow;

    public List<QuestionEntry> Questions { get; set; } = [];
}
```

5.2 Create the question entity

src/QuestionCanvas.Domain/Entities/QuestionEntry.cs

```
namespace QuestionCanvas.Domain.Entities;

public sealed class QuestionEntry
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid ProjectId { get; set; }
    public string QuestionText { get; set; } = string.Empty;
    public string OwnerUserId { get; set; } = string.Empty;
    public DateTimeOffset CreatedUtc { get; set; } = DateTimeOffset.UtcNow;

    public QuestionProject Project { get; set; } = null!;
}
```

Why store OwnerUserId on both records? The project ownership is enough to navigate the relationship, but the question owner gives an explicit audit/security invariant and prepares Part 2 for direct question-level authorization checks. The service will always set the question owner from the authenticated user, never from request JSON.

Checkpoint: Do not continue until every item below is true.

[] Domain builds.

[] No Domain file imports Microsoft.AspNetCore or Microsoft.EntityFrameworkCore.

6. Build the Application layer

The Application project defines the use cases and contracts that the API will call. It is also where we enforce business-level validation a second time, even though controller model validation exists. This protects the use case if another caller is added later.

6.1 Create request and response DTOs

src/QuestionCanvas.Application/Contracts/WorkspaceContracts.cs

```
using System.ComponentModel.DataAnnotations;

namespace QuestionCanvas.Application.Contracts;

public sealed class CreateProjectRequest
{
    [Required]
    [StringLength(120, MinimumLength = 1)]
    public string ProjectName { get; init; } = string.Empty;
}

public sealed class CreateQuestionRequest
{
    [Required]
    [StringLength(2000, MinimumLength = 1)]
    public string Question { get; init; } = string.Empty;
}

public sealed record ProjectSummaryDto(
    Guid Id,
    string ProjectName,
    int QuestionCount,
    DateTimeOffset CreatedUtc);

public sealed record QuestionDto(
    Guid Id,
    string Question,
    DateTimeOffset CreatedUtc);

public sealed record ProjectDetailsDto(
    Guid Id,
    string ProjectName,
    DateTimeOffset CreatedUtc,
    IReadOnlyList<QuestionDto> Questions);
```

6.2 Create application exceptions

src/QuestionCanvas.Application/Exceptions/ApplicationExceptions.cs

```
namespace QuestionCanvas.Application.Exceptions;

public sealed class AppValidationException(string message) : Exception(message);

public sealed class NotFoundException(string message) : Exception(message);

public sealed class ConflictException(string message) : Exception(message);
```

6.3 Create the current-user abstraction

src/QuestionCanvas.Application/Security/ICurrentUserService.cs

```

namespace QuestionCanvas.Application.Security;

public interface ICurrentUserService
{
    string UserId { get; }
}

```

The Application layer needs the current user ID, but it should not depend on HttpContext. The API project will implement this interface from the authenticated claims principal.

6.4 Create the persistence contract

src/QuestionCanvas.Application/Persistence/IQuestionWorkspaceRepository.cs

```

using QuestionCanvas.Domain.Entities;

namespace QuestionCanvas.Application.Persistence;

public interface IQuestionWorkspaceRepository
{
    Task<bool> ProjectNameExistsForUserAsync(
        string ownerUserId,
        string normalizedName,
        CancellationToken cancellationToken);

    Task AddProjectAsync(
        QuestionProject project,
        CancellationToken cancellationToken);

    Task<IReadOnlyList<QuestionProject>> ListOwnedProjectsAsync(
        string ownerUserId,
        CancellationToken cancellationToken);

    Task<QuestionProject?> GetOwnedProjectDetailsAsync(
        Guid projectId,
        string ownerUserId,
        CancellationToken cancellationToken);

    Task<QuestionProject?> GetOwnedProjectForUpdateAsync(
        Guid projectId,
        string ownerUserId,
        CancellationToken cancellationToken);

    Task SaveChangesAsync(CancellationToken cancellationToken);
}

```

6.5 Create the use-case service interface

src/QuestionCanvas.Application/Services/IQuestionWorkspaceService.cs

```

using QuestionCanvas.Application.Contracts;

namespace QuestionCanvas.Application.Services;

public interface IQuestionWorkspaceService
{
    Task<ProjectSummaryDto> CreateProjectAsync(
        CreateProjectRequest request,
        CancellationToken cancellationToken);

    Task<IReadOnlyList<ProjectSummaryDto>> ListProjectsAsync(
        CancellationToken cancellationToken);

    Task<ProjectDetailsDto> GetProjectAsync(
        Guid projectId,
        CancellationToken cancellationToken);

    Task<QuestionDto> AddQuestionAsync(
        Guid projectId,
        CreateQuestionRequest request,
        CancellationToken cancellationToken);
}

```

6.6 Implement the application service

src/QuestionCanvas.Application/Services/QuestionWorkspaceService.cs

```
using QuestionCanvas.Application.Contracts;
using QuestionCanvas.Application.Exceptions;
using QuestionCanvas.Application.Persistence;
using QuestionCanvas.Application.Security;
using QuestionCanvas.Domain.Entities;

namespace QuestionCanvas.Application.Services;

public sealed class QuestionWorkspaceService(
    IQuestionWorkspaceRepository repository,
    ICurrentUserService currentUser) : IQuestionWorkspaceService
{
    public async Task<ProjectSummaryDto> CreateProjectAsync(
        CreateProjectRequest request,
        CancellationToken cancellationToken)
    {
        var projectName = request.ProjectName.Trim();
        if (projectName.Length is < 1 or > 120)
        {
            throw new AppValidationException(
                "Project Name must be between 1 and 120 characters.");
        }

        var normalizedName = projectName.ToUpperInvariant();
        var exists = await repository.ProjectNameExistsForUserAsync(
            currentUser.UserId,
            normalizedName,
            cancellationToken);

        if (exists)
        {
            throw new ConflictException(
                "You already have a project with that name.");
        }

        var project = new QuestionProject
        {
            Name = projectName,
            NormalizedName = normalizedName,
            OwnerUserId = currentUser.UserId
        };

        await repository.AddProjectAsync(project, cancellationToken);
        await repository.SaveChangesAsync(cancellationToken);

        return ToSummary(project);
    }

    public async Task<IReadOnlyList<ProjectSummaryDto>> ListProjectsAsync(
        CancellationToken cancellationToken)
    {
        var projects = await repository.ListOwnedProjectsAsync(
            currentUser.UserId,
            cancellationToken);

        return projects.Select(ToSummary).ToList();
    }

    public async Task<ProjectDetailsDto> GetProjectAsync(
        Guid projectId,
        CancellationToken cancellationToken)
    {
        var project = await repository.GetOwnedProjectDetailsAsync(
            projectId,
            currentUser.UserId,
            cancellationToken);

        if (project is null)
        {
            throw new NotFoundException("Project was not found.");
        }

        return new ProjectDetailsDto(
            project.Id,
            project.Name,
            project.CreatedUtc,
            project.Questions
        );
    }
}
```

```

        .OrderByDescending(q => q.CreatedUtc)
        .Select(q => new QuestionDto(q.Id, q.QuestionText, q.CreatedUtc))
        .ToList());
    }

    public async Task<QuestionDto> AddQuestionAsync(
        Guid projectId,
        CreateQuestionRequest request,
        CancellationToken cancellationToken)
    {
        var questionText = request.Question.Trim();
        if (questionText.Length is < 1 or > 2000)
        {
            throw new AppValidationException(
                "Question must be between 1 and 2000 characters.");
        }

        var project = await repository.GetOwnedProjectForUpdateAsync(
            projectId,
            currentUser.UserId,
            cancellationToken);

        if (project is null)
        {
            throw new NotFoundException("Project was not found.");
        }

        var question = new QuestionEntry
        {
            ProjectId = project.Id,
            QuestionText = questionText,
            OwnerUserId = currentUser.UserId
        };

        project.Questions.Add(question);
        await repository.SaveChangesAsync(cancellationToken);

        return new QuestionDto(
            question.Id,
            question.QuestionText,
            question.CreatedUtc);
    }

    private static ProjectSummaryDto ToSummary(QuestionProject project) =>
        new(
            project.Id,
            project.Name,
            project.Questions.Count,
            project.CreatedUtc);
}

```

Security pattern: Notice that no create request contains OwnerUserId. Ownership always comes from ICurrentUserService. A client cannot claim another user ID in JSON.

Checkpoint: Do not continue until every item below is true.

- [] Application builds.
- [] Application has no reference to HttpContext, IdentityUser, or SQLite.
- [] The service checks project ownership through repository methods that require the authenticated user ID.

7. Build the Infrastructure layer with EF Core and Identity

7.1 Create the Identity user type

src/QuestionCanvas.Infrastructure/Identity/ApplicationUser.cs

```

using Microsoft.AspNetCore.Identity;

namespace QuestionCanvas.Infrastructure.Identity;

```

```
public sealed class ApplicationUser : IdentityUser
{
}
}
```

7.2 Create AppDbContext

src/QuestionCanvas.Infrastructure/Persistence/AppDbContext.cs

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using QuestionCanvas.Domain.Entities;
using QuestionCanvas.Infrastructure.Identity;

namespace QuestionCanvas.Infrastructure.Persistence;

public sealed class AppDbContext(DbContextOptions<AppDbContext> options)
    : IdentityDbContext<ApplicationUser>(options)
{
    public DbSet<QuestionProject> QuestionProjects => Set<QuestionProject>();
    public DbSet<QuestionEntry> QuestionEntries => Set<QuestionEntry>();

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        builder.Entity<QuestionProject>(entity =>
        {
            entity.ToTable("QuestionProjects");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.Name)
                .HasMaxLength(120)
                .IsRequired();

            entity.Property(x => x.NormalizedName)
                .HasMaxLength(120)
                .IsRequired();

            entity.Property(x => x.OwnerUserId)
                .HasMaxLength(450)
                .IsRequired();

            entity.HasIndex(x => new { x.OwnerUserId, x.NormalizedName })
                .IsUnique();

            entity.HasMany(x => x.Questions)
                .WithOne(x => x.Project)
                .HasForeignKey(x => x.ProjectId)
                .OnDelete(DeleteBehavior.Cascade);

            entity.HasOne<ApplicationUser>()
                .WithMany()
                .HasForeignKey(x => x.OwnerUserId)
                .OnDelete(DeleteBehavior.Cascade);
        });

        builder.Entity<QuestionEntry>(entity =>
        {
            entity.ToTable("QuestionEntries");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.QuestionText)
                .HasMaxLength(2000)
                .IsRequired();

            entity.Property(x => x.OwnerUserId)
                .HasMaxLength(450)
                .IsRequired();

            entity.HasIndex(x => new { x.ProjectId, x.CreatedUtc });

            entity.HasOne<ApplicationUser>()
                .WithMany()
                .HasForeignKey(x => x.OwnerUserId)
                .OnDelete(DeleteBehavior.Cascade);
        });
    }
}
```

Enterprise points in this mapping: field lengths are enforced in the database model; duplicate project names are prevented per user with a unique composite index; deleting a user can cascade owned content; and deleting a project cascades its questions.

7.3 Implement the repository and practice LINQ

src/QuestionCanvas.Infrastructure/Persistence/QuestionWorkspaceRepository.cs

```
using Microsoft.EntityFrameworkCore;
using QuestionCanvas.Application.Persistence;
using QuestionCanvas.Domain.Entities;

namespace QuestionCanvas.Infrastructure.Persistence;

public sealed class QuestionWorkspaceRepository(AppDbContext dbContext)
    : IQuestionWorkspaceRepository
{
    public Task<bool> ProjectNameExistsForUserAsync(
        string ownerId,
        string normalizedName,
        CancellationToken cancellationToken) =>
        dbContext.QuestionProjects.AnyAsync(
            p => p.OwnerUserId == ownerId &&
                p.NormalizedName == normalizedName,
            cancellationToken);

    public async Task AddProjectAsync(
        QuestionProject project,
        CancellationToken cancellationToken)
    {
        await dbContext.QuestionProjects.AddAsync(project, cancellationToken);
    }

    public async Task<IReadOnlyList<QuestionProject>> ListOwnedProjectsAsync(
        string ownerId,
        CancellationToken cancellationToken) =>
        await dbContext.QuestionProjects
            .AsNoTracking()
            .Where(p => p.OwnerUserId == ownerId)
            .Include(p => p.Questions)
            .OrderByDescending(p => p.CreatedUtc)
            .ToListAsync(cancellationToken);

    public Task<QuestionProject?> GetOwnedProjectDetailsAsync(
        Guid projectId,
        string ownerId,
        CancellationToken cancellationToken) =>
        dbContext.QuestionProjects
            .AsNoTracking()
            .Where(p => p.Id == projectId && p.OwnerUserId == ownerId)
            .Include(p => p.Questions)
            .SingleOrDefaultAsync(cancellationToken);

    public Task<QuestionProject?> GetOwnedProjectForUpdateAsync(
        Guid projectId,
        string ownerId,
        CancellationToken cancellationToken) =>
        dbContext.QuestionProjects
            .Where(p => p.Id == projectId && p.OwnerUserId == ownerId)
            .SingleOrDefaultAsync(cancellationToken);

    public async Task SaveChangesAsync(CancellationToken cancellationToken)
    {
        await dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

7.4 Understand the LINQ choices

LINQ/EF call	Why it is used
AnyAsync	Asks the database whether a matching row exists without loading an entity.
Where	Pushes the ownership predicate into SQL. This is the critical object-level authorization filter.


```

        - =>
            (StatusCodes.Status500InternalServerError, "Server error")
    };

    if (status >= 500)
    {
        logger.LogError(exception,
            "Unhandled exception. TraceId: {TraceId}",
            httpContext.TraceIdentifier);
    }
    else
    {
        logger.LogWarning(exception,
            "Request failed with status {StatusCode}. TraceId: {TraceId}",
            status,
            httpContext.TraceIdentifier);
    }

    httpContext.Response.StatusCode = status;

    var problem = new ProblemDetails
    {
        Status = status,
        Title = title,
        Detail = status >= 500
            ? "An unexpected error occurred."
            : exception.Message,
        Instance = httpContext.Request.Path
    };

    problem.Extensions["traceId"] = httpContext.TraceIdentifier;

    await problemDetailsService.WriteAsync(new ProblemDetailsContext
    {
        HttpContext = httpContext,
        ProblemDetails = problem,
        Exception = exception
    });

    return true;
}
}

```

Information disclosure: The 500 response does not return the exception message or stack trace. Detailed exceptions belong in server logs, not in production API responses.

8.3 Create the Projects controller

src/QuestionCanvas.Api/Controllers/ProjectsController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
using QuestionCanvas.Application.Contracts;
using QuestionCanvas.Application.Services;

namespace QuestionCanvas.Api.Controllers;

[ApiController]
[Route("api/projects")]
[Authorize]
[EnableRateLimiting("authenticated-api")]
public sealed class ProjectsController(IQuestionWorkspaceService service)
    : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IReadOnlyList<ProjectSummaryDto>>> List(
        CancellationToken cancellationToken)
    {
        var projects = await service.ListProjectsAsync(cancellationToken);
        return Ok(projects);
    }

    [HttpGet("{projectId:guid}")]
    public async Task<ActionResult<ProjectDetailsDto>> GetById(
        Guid projectId,

```

```

        CancellationToken cancellationToken)
    {
        var project = await service.GetProjectAsync(
            projectId,
            cancellationToken);
        return Ok(project);
    }

    [HttpPost]
    public async Task<ActionResult<ProjectSummaryDto>> Create(
        [FromBody] CreateProjectRequest request,
        CancellationToken cancellationToken)
    {
        var project = await service.CreateProjectAsync(
            request,
            cancellationToken);

        return CreatedAtAction(
            nameof(GetById),
            new { projectId = project.Id },
            project);
    }

    [HttpPost("{projectId:guid}/questions")]
    public async Task<ActionResult<QuestionDto>> AddQuestion(
        Guid projectId,
        [FromBody] CreateQuestionRequest request,
        CancellationToken cancellationToken)
    {
        var question = await service.AddQuestionAsync(
            projectId,
            request,
            cancellationToken);

        return Created(
            $"/api/projects/{projectId}/questions/{question.Id}",
            question);
    }
}

```

8.4 Replace Program.cs

Open `src/QuestionCanvas.Api/Program.cs` and replace the whole file. This is the composition root: it wires abstractions to implementations and defines the middleware order.

`src/QuestionCanvas.Api/Program.cs`

```

using System.Threading.RateLimiting;
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using QuestionCanvas.Api.Errors;
using QuestionCanvas.Api.Security;
using QuestionCanvas.Application.Persistence;
using QuestionCanvas.Application.Security;
using QuestionCanvas.Application.Services;
using QuestionCanvas.Infrastructure.Identity;
using QuestionCanvas.Infrastructure.Persistence;

var builder = WebApplication.CreateBuilder(args);

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException(
        "Connection string 'DefaultConnection' was not found.");

var frontendOrigin = builder.Configuration["Frontend:Origin"]
    ?? throw new InvalidOperationException(
        "Configuration value 'Frontend:Origin' was not found.");

builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(connectionString));

builder.Services.AddAuthorization();

```

```

builder.Services
    .AddIdentityApiEndpoints<ApplicationUser>(options =>
    {
        options.User.RequireUniqueEmail = true;
        options.Password.RequiredLength = 12;
        options.Password.RequireDigit = true;
        options.Password.RequireLowercase = true;
        options.Password.RequireUppercase = true;
        options.Password.RequireNonAlphanumeric = true;
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(10);
    })
    .AddEntityFrameworkStores<AppDbContext>();

builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserService, CurrentUserService>();
builder.Services.AddScoped<IQuestionWorkspaceRepository, QuestionWorkspaceRepository>();
builder.Services.AddScoped<IQuestionWorkspaceService, QuestionWorkspaceService>();

builder.Services.AddCors(options =>
{
    options.AddPolicy("frontend", policy =>
        policy.WithOrigins(frontendOrigin)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials());
});

builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    options.AddFixedWindowLimiter("auth", limiter =>
    {
        limiter.PermitLimit = 20;
        limiter.Window = TimeSpan.FromMinutes(1);
        limiter.QueueLimit = 0;
    });

    options.AddPolicy("authenticated-api", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.User.Identity?.Name
                ?? httpContext.Connection.RemoteIpAddress?.ToString()
                ?? "anonymous",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 120,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 0,
                AutoReplenishment = true
            }
        ));
});

builder.Services.AddHealthChecks()
    .AddDbContextCheck<AppDbContext>("sqlite", tags: ["ready"]);

var app = builder.Build();

app.UseExceptionHandler();
app.UseStatusCodePages();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();
app.UseCors("frontend");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();

var auth = app.MapGroup("/api/auth")
    .RequireRateLimiting("auth");

auth.MapIdentityApi<ApplicationUser>();

auth.MapPost("/logout", async (SignInManager<ApplicationUser> signInManager) =>
{
    await signInManager.SignOutAsync();
    return Results.NoContent();
});

```

```

}).RequireAuthorization();

auth.MapGet("/session", (HttpContext context) =>
    Results.Ok(new
    {
        email = context.User.Identity?.Name
    }))
    .RequireAuthorization();

app.MapControllers();

app.MapHealthChecks("/health/live", new HealthCheckOptions
{
    Predicate = _ => false
});

app.MapHealthChecks("/health/ready", new HealthCheckOptions
{
    Predicate = check => check.Tags.Contains("ready")
});

app.Run();

public partial class Program { }

```

8.5 Configure appsettings.json

src/QuestionCanvas.Api/appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=questioncanvas.db"
  },
  "Frontend": {
    "Origin": "http://localhost:5173"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

8.6 Make the local API ports deterministic

Edit the https launch profile in src/QuestionCanvas.Api/Properties/launchSettings.json so the applicationUrl is exactly:

Inside the HTTPS project profile

```
"applicationUrl": "https://localhost:7043;http://localhost:5043"
```

Keep the rest of the generated launchSettings.json structure. The React lesson will proxy /api to https://localhost:7043, so using a fixed port prevents setup drift between students.

Checkpoint: Do not continue until every item below is true.

- [] dotnet build QuestionCanvas.slnx succeeds.
- [] Program.cs includes UseAuthentication before UseAuthorization.
- [] CORS allows only the configured frontend origin and allows credentials.
- [] ProjectsController has both [Authorize] and a rate-limit policy.

9. Create the SQLite database with migrations

Do not call Database.EnsureCreated in an application that uses migrations. Migrations create an auditable schema history and are the path that can later be promoted to SQL Server/PostgreSQL or deployment automation.

macOS/Linux terminal

```
dotnet ef migrations add InitialCreate \  
  --project src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj \  
  --startup-project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj \  
  --output-dir Persistence/Migrations
```

PowerShell

```
dotnet ef migrations add InitialCreate \  
  --project src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj \  
  --startup-project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj \  
  --output-dir Persistence/Migrations
```

macOS/Linux terminal

```
dotnet ef database update \  
  --project src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj \  
  --startup-project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
```

PowerShell

```
dotnet ef database update \  
  --project src/QuestionCanvas.Infrastructure/QuestionCanvas.Infrastructure.csproj \  
  --startup-project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
```

The SQLite file is created relative to the API working directory when the app or tooling starts. In VS Code you should find questioncanvas.db under src/QuestionCanvas.Api after the database update.

Open the generated migration. Confirm that it contains ASP.NET Identity tables plus QuestionProjects and QuestionEntries, a foreign key from QuestionEntries to QuestionProjects, owner foreign keys, and the unique owner/name index.

Checkpoint: Do not continue until every item below is true.

- [] A migration folder exists under Infrastructure/Persistence/Migrations.
- [] questioncanvas.db exists.
- [] dotnet ef database update completes without errors.

10. Run and test authentication and ownership

10.1 Start the API

Run from backend

```
dotnet run --project src/QuestionCanvas.Api/QuestionCanvas.Api.csproj
```

Leave that terminal running. Open a second VS Code terminal for curl commands.

10.2 Verify health checks

macOS/Linux

```
curl -k https://localhost:7043/health/live  
curl -k https://localhost:7043/health/ready
```

Windows

```
curl.exe -k https://localhost:7043/health/live  
curl.exe -k https://localhost:7043/health/ready
```

Both should return Healthy. The live endpoint proves the process can answer; the ready endpoint also checks the EF Core DbContext can reach SQLite.

10.3 Register and sign in as Alice

macOS/Linux

```
curl -k -X POST "https://localhost:7043/api/auth/register" \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","password":"Canvas!Pass123"}'

curl -k -c alice.cookies -b alice.cookies \
-X POST "https://localhost:7043/api/auth/login?useCookies=true" \
-H "Content-Type: application/json" \
-d '{"email":"alice@example.com","password":"Canvas!Pass123"}'
```

Windows

```
curl.exe -k -X POST "https://localhost:7043/api/auth/register" -H "Content-Type: application/json" -d
"{\"email\":\"alice@example.com\",\"password\":\"Canvas!Pass123\"}"

curl.exe -k -c alice.cookies -b alice.cookies -X POST "https://localhost:7043/api/auth/login?useCookies=true"
-H "Content-Type: application/json" -d "{\"email\":\"alice@example.com\",\"password\":\"Canvas!Pass123\"}"
```

Verify Alice session (use curl.exe on Windows)

```
curl -k -b alice.cookies https://localhost:7043/api/auth/session
```

10.4 Create a project and save its ID

macOS/Linux

```
curl -k -b alice.cookies \
-X POST "https://localhost:7043/api/projects" \
-H "Content-Type: application/json" \
-d '{"projectName":"Bird Study"}'
```

Windows

```
curl.exe -k -b alice.cookies -X POST "https://localhost:7043/api/projects" -H "Content-Type: application/json"
-d "{\"projectName\":\"Bird Study\"}"
```

Copy the returned id GUID; the next examples call it PROJECT_ID.

10.5 Add multiple questions to the same project

macOS/Linux; replace PROJECT_ID

```
curl -k -b alice.cookies \
-X POST "https://localhost:7043/api/projects/PROJECT_ID/questions" \
-H "Content-Type: application/json" \
-d '{"question":"What visual pattern should this answer produce?}"'

curl -k -b alice.cookies \
-X POST "https://localhost:7043/api/projects/PROJECT_ID/questions" \
-H "Content-Type: application/json" \
-d '{"question":"How should the second generated image differ?}"'
```

Use curl.exe and escaped JSON on Windows, as in the prior commands.

10.6 Read the project and observe the one-to-many relationship

List and detail endpoints

```
curl -k -b alice.cookies https://localhost:7043/api/projects
curl -k -b alice.cookies https://localhost:7043/api/projects/PROJECT_ID
```

The project details response should contain both saved questions. Restart the API and call the endpoints again; the data should still be present, proving SQLite persistence rather than in-memory storage.

10.7 Prove object-level authorization with a second user

Register Bob, log in to a separate cookie jar, then request Alice's PROJECT_ID. This is the security test that matters most for the ownership requirement.

macOS/Linux

```
curl -k -X POST "https://localhost:7043/api/auth/register" \
-H "Content-Type: application/json" \
-d '{"email":"bob@example.com","password":"Canvas!Pass123"}'

curl -k -c bob.cookies -b bob.cookies \
-X POST "https://localhost:7043/api/auth/login?useCookies=true" \
-H "Content-Type: application/json" \
-d '{"email":"bob@example.com","password":"Canvas!Pass123"}'

curl -k -i -b bob.cookies \
"https://localhost:7043/api/projects/PROJECT_ID"
```

Expected result: 404. We intentionally do not return 403 for an object another user does not own; 404 avoids confirming that the object exists.

10.8 Trigger enterprise error responses

- Create the same project name twice as Alice: expect 409 with ProblemDetails.
- Send an empty Project Name: expect 400 validation response.
- Call /api/projects without a cookie: expect 401.
- Use a random project GUID while authenticated: expect 404 ProblemDetails.
- Rapidly exceed the configured request limit: expect 429.

Checkpoint: Do not continue until every item below is true.

- [] Alice can create one project with multiple questions.
- [] Data survives an API restart.
- [] Bob cannot read Alice's project ID and receives 404.
- [] Duplicate project name returns 409.
- [] Unauthenticated project request returns 401.

11. Add integration-test scaffolding

The first integration tests verify application bootstrapping, liveness, and authentication enforcement. Later parts can add a temporary SQLite database and full authenticated user A/user B tests.

tests/QuestionCanvas.Api.IntegrationTests/BasicApiTests.cs

```
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;

namespace QuestionCanvas.Api.IntegrationTests;

public sealed class BasicApiTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public BasicApiTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task LiveHealthEndpoint_ReturnsSuccess()
    {
        var response = await _client.GetAsync("/health/live");
        response.EnsureSuccessStatusCode();
    }
}
```

```
[Fact]
public async Task ProjectsEndpoint_WithoutAuthentication_ReturnsUnauthorized()
{
    var response = await _client.GetAsync("/api/projects");

    Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);
}
}
```

Run all tests

```
dotnet test QuestionCanvas.slnx
```

Why public partial Program exists: WebApplicationFactory needs a stable entry-point type. The public partial Program declaration at the end of Program.cs exposes the top-level application entry point to the integration-test project.

11.1 Final quality commands

Run before considering Part 1 complete

```
dotnet format QuestionCanvas.slnx
dotnet build QuestionCanvas.slnx --configuration Release
dotnet test QuestionCanvas.slnx --configuration Release
```

Checkpoint: Do not continue until every item below is true.

- [] dotnet format completes.
- [] Release build succeeds.
- [] All tests pass.

12. Architecture review and Part 2 handoff

12.1 Request flow

1. React sends a credentialed HTTPS request.
2. ASP.NET Core Identity authenticates the cookie and builds the ClaimsPrincipal.
3. [Authorize] blocks anonymous access before the project use case runs.
4. CurrentUserService converts the NameIdentifier claim into the application UserId abstraction.
5. QuestionWorkspaceService validates the use case and supplies the authenticated user ID to the repository.
6. QuestionWorkspaceRepository uses LINQ with both object ID and owner ID in the SQL predicate.
7. AppDbContext tracks writes and SaveChangesAsync commits them to SQLite.
8. Exceptions become ProblemDetails through GlobalExceptionHandler; unexpected details remain in server logs.

12.2 What Part 2 should add without changing Part 1

- A generation service interface in Application (for example IQuestionImageGenerator).
- An Infrastructure implementation of the generation algorithm or external model client.
- A protected endpoint scoped beneath both project ID and question ID.
- A question lookup that filters by project ID, question ID, and OwnerUserId before any generation work occurs.
- Binary image/png output with appropriate cache/security headers.
- CancellationToken propagation into expensive generation work.
- Rate limiting/concurrency controls for the expensive generation endpoint.
- Integration tests proving another user cannot generate from someone else's saved question.

12.3 Enterprise features intentionally deferred

SQLite is appropriate for the learning project and local persistence, but a horizontally scaled production service would normally move to a server database. Deployment secrets, external identity providers/OIDC, OpenTelemetry, distributed

tracing/exporters, containerization, CI/CD, cloud database resilience, and production secret stores are excellent later lessons. The current architecture leaves clear insertion points for them.

Appendix A. Endpoint reference

Method	Path	Auth	Purpose
POST	/api/auth/register	No	Create Identity user.
POST	/api/auth/login?useCookies=true	No	Create browser auth cookie.
POST	/api/auth/logout	Yes	Clear auth session.
GET	/api/auth/session	Yes	Return current user session summary.
GET	/api/projects	Yes	List only current user projects.
POST	/api/projects	Yes	Create project owned by current user.
GET	/api/projects/{projectId}	Yes	Get owned project plus questions.
POST	/api/projects/{projectId}/questions	Yes	Create question under owned project.
GET	/health/live	No	Process liveness.
GET	/health/ready	No	Readiness including EF/SQLite.
GET	/openapi/v1.json (Development)	No	Generated OpenAPI document from built-in ASP.NET Core OpenAPI support.

Appendix B. Troubleshooting

Symptom	Likely cause	Fix
QuestionCanvas.sln is missing	.NET 10 defaults to .slnx	Use QuestionCanvas.slnx exactly as shown.
dotnet ef is not found	Local tool not restored	Run dotnet tool restore from backend.
Migration cannot create DbContext	Startup project or package mismatch	Use both --project Infrastructure and --startup-project Api; keep EF packages on 10.0.x.
React later gets 401 after login	Cookies not included or proxy/origin mismatch	Frontend fetch must use credentials: include; use the Vite /api proxy from the React lesson.
CORS blocks browser request	Origin differs from appsettings	Use http://localhost:5173 exactly during the Vite lesson or update Frontend:Origin deliberately.
HTTPS certificate warning	Development cert not trusted	Run dotnet dev-certs https --trust and restart browser/terminal.
Bob can read Alice data	Ownership predicate omitted	Every object query must include OwnerUserId from ICurrentUserService, never a user ID from the request.
Duplicate project causes unexpected DB error	Race between precheck and insert	The unique DB index is the final protection. A later hardening step can translate the provider-specific unique-constraint exception into 409.
Strict 500 details are not visible	By design	Use server logs and traceId; do not expose stack traces to clients.

Reference baseline

- Microsoft .NET support policy: .NET 10 is LTS and active in August 2026.
- Microsoft Learn: Use Identity to secure a Web API backend for SPAs.
- Microsoft Learn: Handle errors in ASP.NET Core APIs / ProblemDetails.
- Microsoft Learn: ASP.NET Core integration tests with WebApplicationFactory.
- Microsoft Learn: Health checks in ASP.NET Core.
- Microsoft Learn: Rate limiting middleware in ASP.NET Core.
- Microsoft Learn: EF Core CLI tools and migrations.